

GitHub Source Code Link:

https://github.com/chirayus20/Jenkins_CI_CD_Chirayu

Part 1: Deploy Flask and Express on a Single EC2 Instance

Objective: Deploy both the Flask backend and the Express frontend on a single Amazon EC2 instance.

Provisioning the EC2 Instance

1. Launch an EC2 instance on AWS (a free-tier eligible instance is fine).
2. SSH into the instance and install the dependencies: Python (for Flask), Node.js (for Express), and Git (for pulling the application code).

Application Setup

3. Clone the Flask and Express repositories onto the EC2 instance.
4. Install the required dependencies for both applications using `pip` and `npm`.
5. Configure both applications to run on different ports (e.g., Flask on port 5000 and Express on port 3000).
6. Start the applications using process managers like `pm2` or `systemd` to ensure they remain active.

Deliverables

7. A running EC2 instance with Flask and Express accessible via the instance's public IP.
8. A description or diagram of the deployed architecture.

Part 2: Implement CI/CD Pipeline Using Jenkins

Objective: Automate the deployment of the Flask and Express applications using Jenkins.

Install Jenkins

9. Install Jenkins on the same EC2 instance or on a separate machine.
10. **Configure Jenkins** by installing essential plugins like Git, NodeJS, and Python.

Set Up Jenkins Pipeline

11. Create two separate Jenkins pipelines for the Flask and Express applications.
12. **Pipeline Steps:** Pull the latest code from the respective Git repositories.
13. Install dependencies for Flask (`pip install -r requirements.txt`) and Express (`npm install`).
14. Restart the applications using the process manager (e.g., `pm2 restart`).

Triggering the Pipeline

15. Set up a GitHub webhook to trigger the Jenkins pipeline on every push to the repositories.

Optional Enhancements

16. Add testing stages to the pipeline for both applications.
17. Configure environment variables in Jenkins for managing sensitive data like API keys.

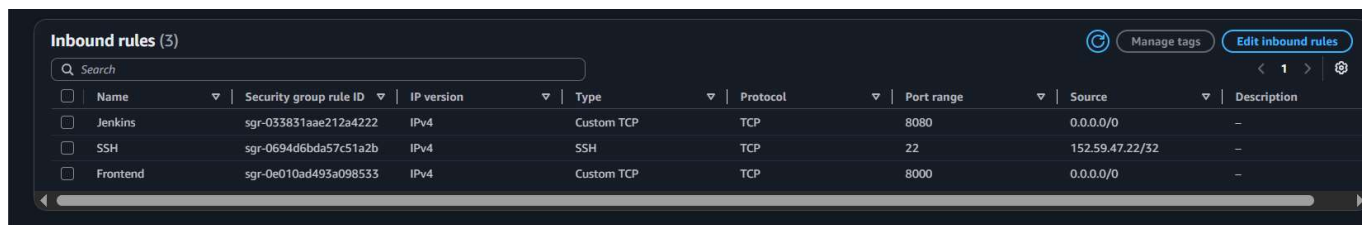
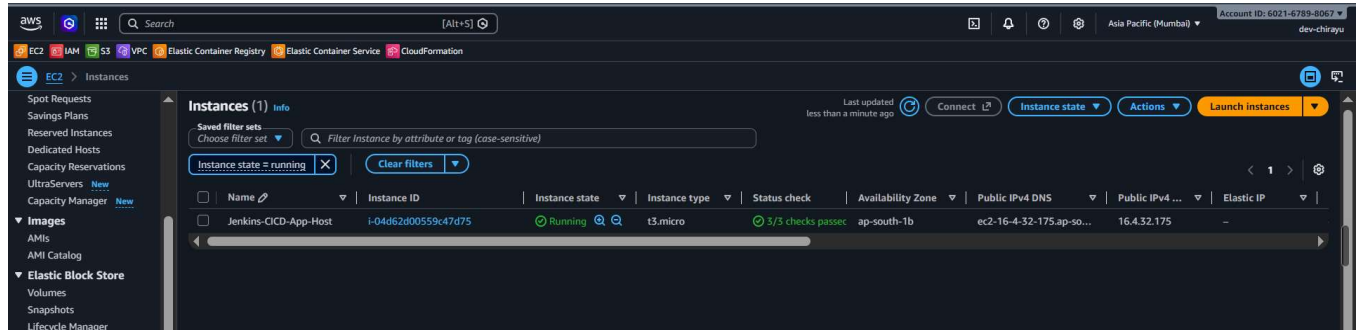
Deliverables

18. A fully functional CI/CD pipeline that automates the deployment process for Flask and Express.
19. A Jenkins pipeline script (`Jenkinsfile`) for each application.
20. Evidence of the pipeline working (e.g., screenshots of successful builds and deployments).

Part 1: Deploy Flask and Express on a Single EC2 Instance.

Explanation: Part 1

Step 1: EC2 Instance Launch & Security Group Configuration.



Step 2: Server Dependencies Installation.

We will SSH into the EC2 instance and run the following commands:

1. Update system packages

```
sudo apt update && sudo apt upgrade -y
```

2. Install Git and build tools

```
sudo apt install -y git curl wget build-essential
```

3. Install Python 3 and pip (Flask backend)

```
sudo apt install -y python3 python3-pip python3-venv
```

4. Install Node.js and PM2 (Express frontend)

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

```
sudo npm install -g pm2
```

- Launched a t3.micro Ubuntu EC2 instance on AWS, connected over SSH, and installed Python, Node.js, Git, and PM2.
- Cloned the repository, installed dependencies with pip (in a virtual environment) and npm, and configured Flask on port 9000 and Express on port 8000.
- Started both apps with PM2 so they stay running, and opened ports 8000 (Express) and 9000 (Flask) in the Security Group to reach them at <http://13.203.220.158:8000/>

```
→ Jenkins_CICD_Chirayu git:(main → git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        backend/Jenkinsfile
        frontend/Jenkinsfile

nothing added to commit but untracked files present (use "git add" to track)
→ Jenkins_CICD_Chirayu git:(main → git add .
→ Jenkins_CICD_Chirayu git:(main → git commit -m "Add Jenkinsfiles for Flask and Express applications"
[main ffa9762] Add Jenkinsfiles for Flask and Express applications
 2 files changed, 117 insertions(+)
 create mode 100644 backend/Jenkinsfile
 create mode 100644 frontend/Jenkinsfile
→ Jenkins_CICD_Chirayu git:(main → git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.32 KiB | 450.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To github.com:chirayus20/Jenkins_CICD_Chirayu.git
 a78479c..ffa9762  main → main
→ Jenkins_CICD_Chirayu git:(main → |
```

Part 2: Implement CI/CD Pipeline Using Jenkins

5. Install Java and Jenkins

```
sudo apt install -y fontconfig openjdk-21-jre
```

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]  
https://pkg.jenkins.io/debian-stable binary/" | sudo tee  
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install -y jenkins
```

6. Start Jenkins

```
sudo systemctl enable jenkins
```

```
sudo systemctl start jenkins
```

7. Initial admin password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

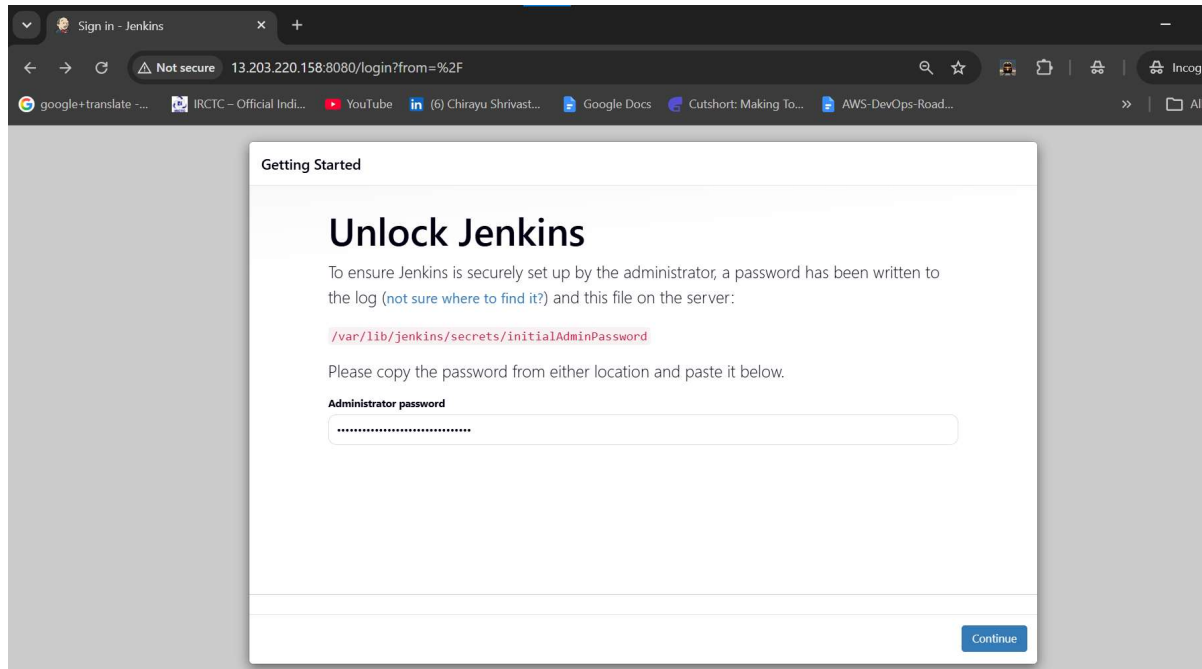
Step 3: Jenkins is active (running) perfectly. Sudo access has also been set up.

```
ubuntu@ip-172-31-2-191: ~/jenkins_CICD_Chirayu$ echo "jenkins ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/jenkins
jenkins ALL=(ALL) NOPASSWD: ALL
ubuntu@ip-172-31-2-191:~/jenkins_CICD_Chirayu$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 07:33:46 UTC; 2h 53min ago
     Main PID: 35167 (java)
       Tasks: 37 (limit: 1002)
      Memory: 267.9M (peak: 317.0M swap: 111.6M swap peak: 136.6M)
         CPU: 41.948s
    CGroup: /system.slice/jenkins.service
            └─35167 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

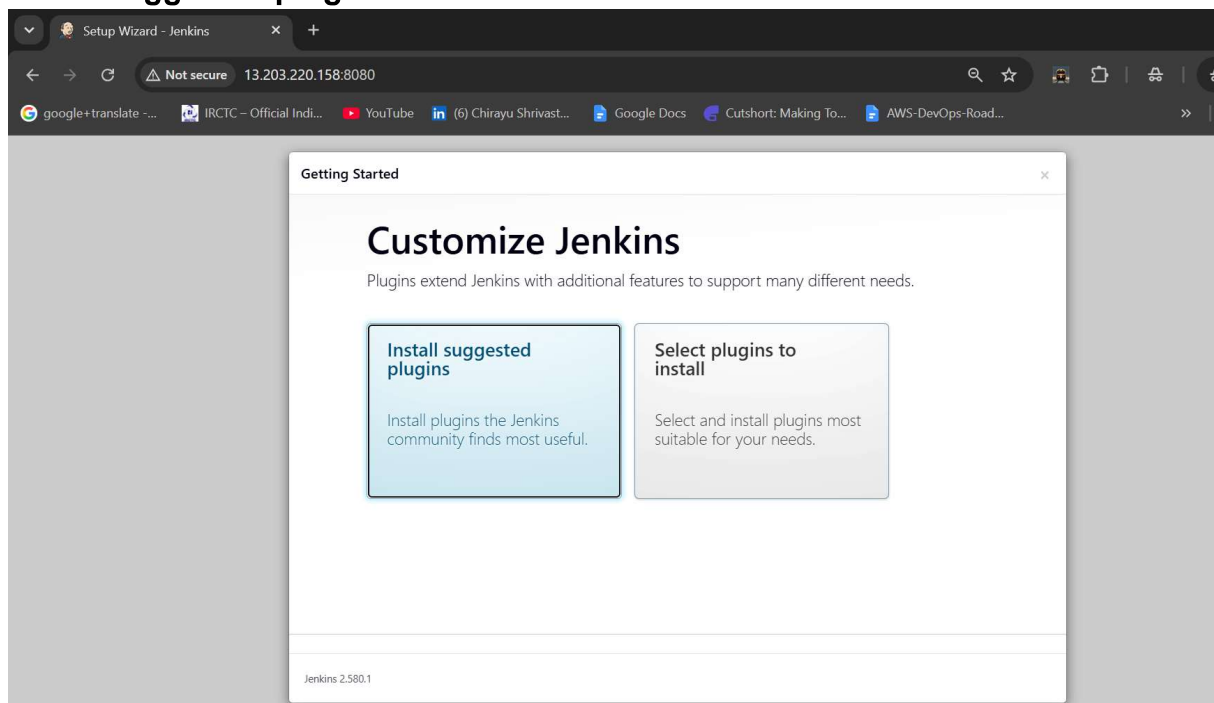
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]>
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:39 ip-172-31-2-191 jenkins[35167]: [LF]> *****
Oct 08 07:33:46 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:46.901+0000 [id=31] INFO jenkins.InitReactorRunner$1onAttained: Completed initialization
Oct 08 07:33:46 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:46.936+0000 [id=24] INFO hudson.LifecycleLifecycle#onReady: Jenkins is fully up and running
Oct 08 07:33:46 ip-172-31-2-191 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 08 07:33:48 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:48.873+0000 [id=50] INFO h.n.DownloadService$DownloadableLoad: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
Oct 08 07:33:48 ip-172-31-2-191 jenkins[35167]: 2026-10-08 07:33:48.875+0000 [id=50] INFO hudson.util.Retrier$Start: Performed the action check updates server successfully at the attempt #1
ubuntu@ip-172-31-2-191:~/jenkins_CICD_Chirayu$
```

CI/CD Pipelines (Jenkins):

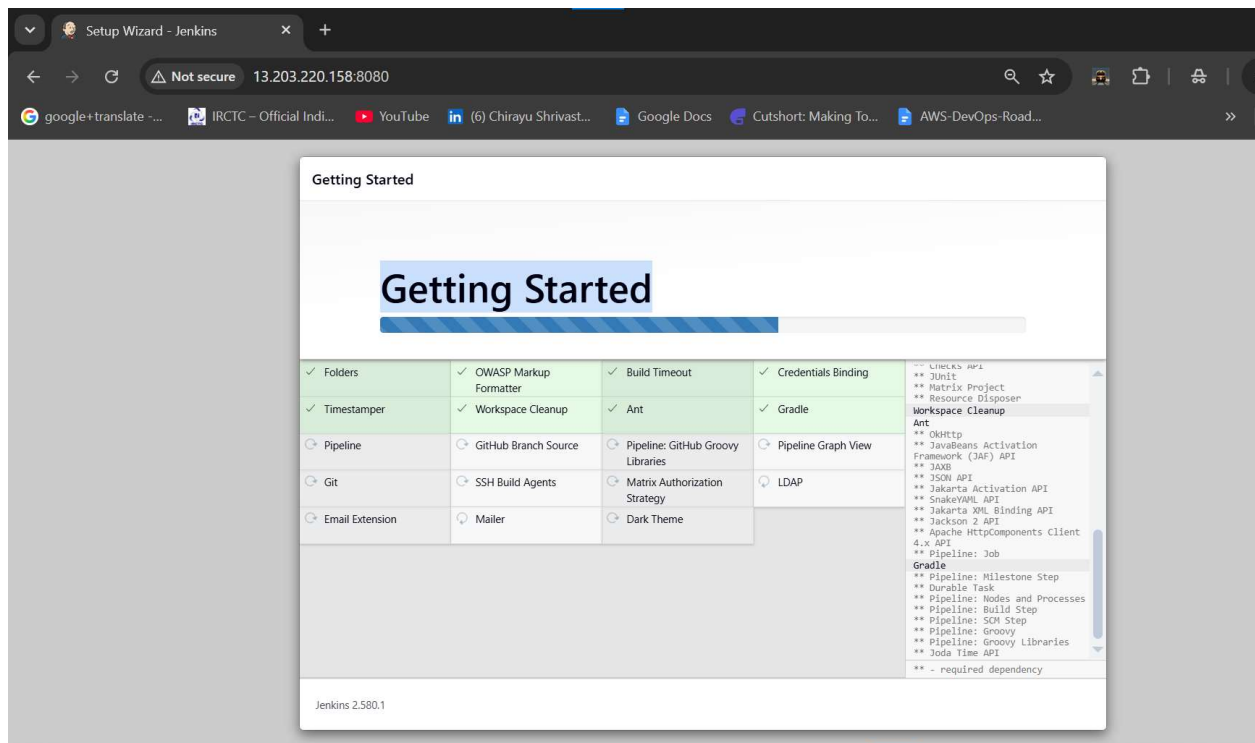
sudo cat /var/lib/jenkins/secrets/initialAdminPassword:



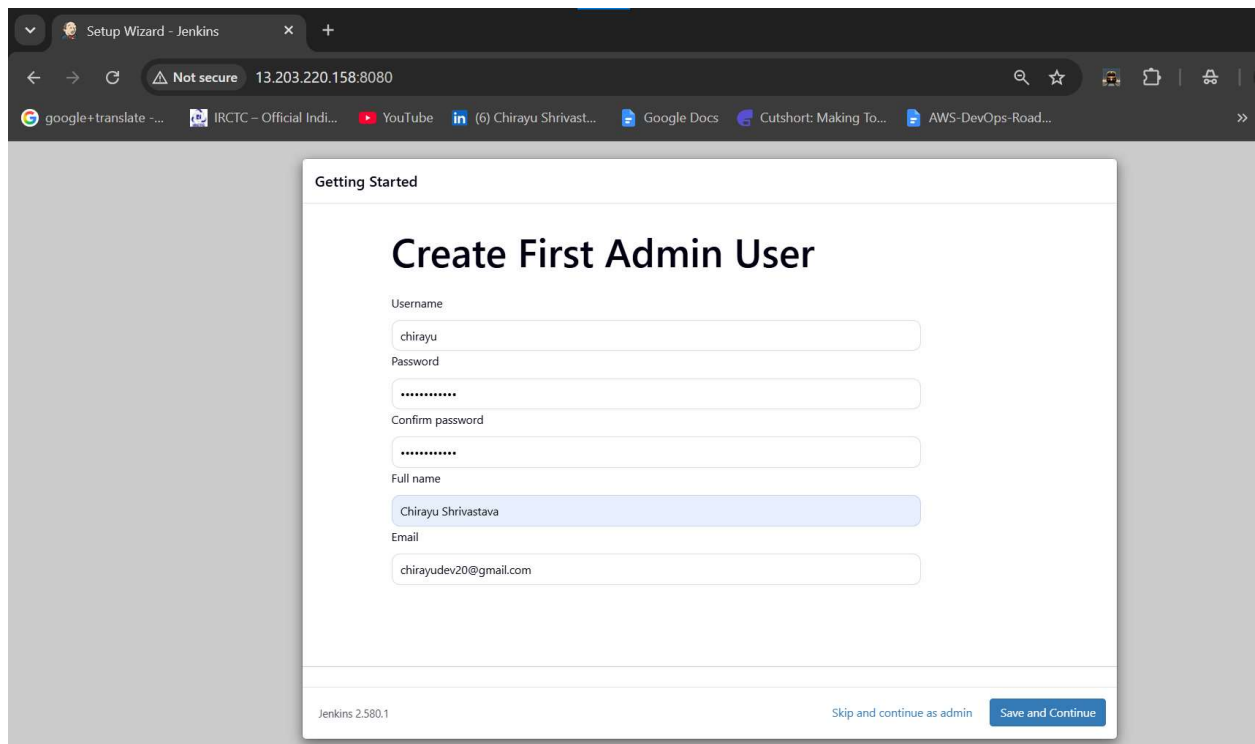
Install suggested plugins :

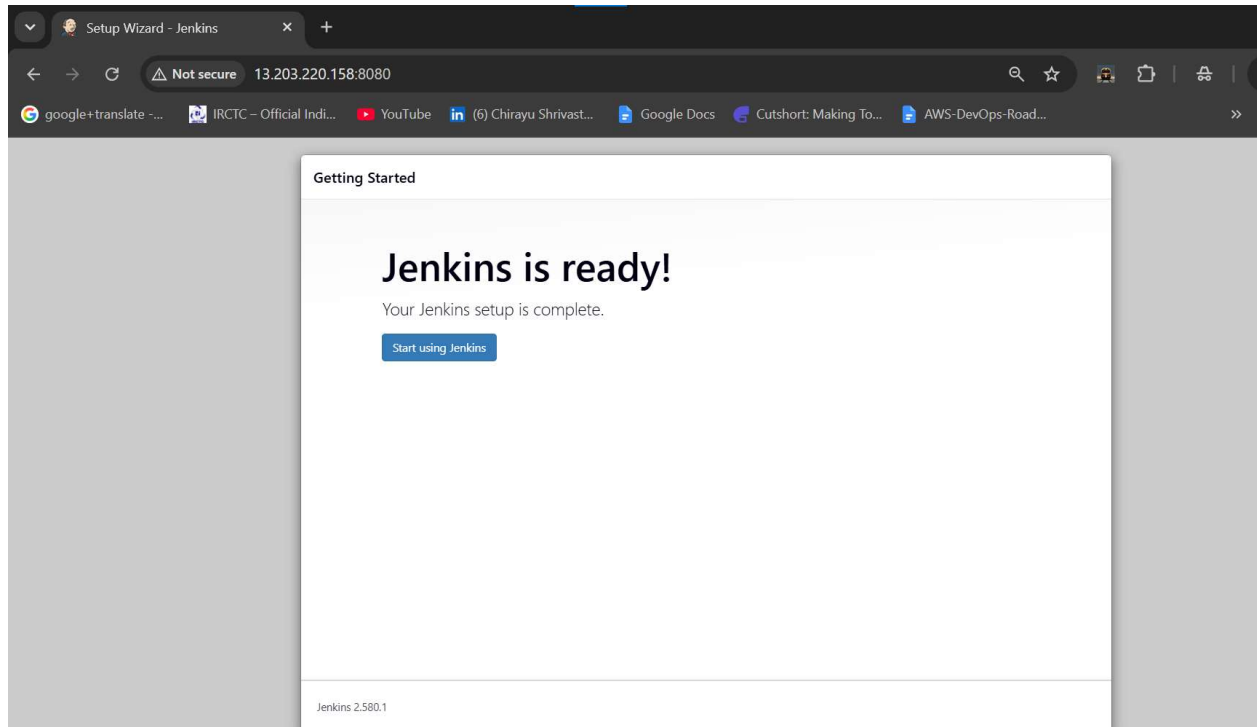


Getting Started:

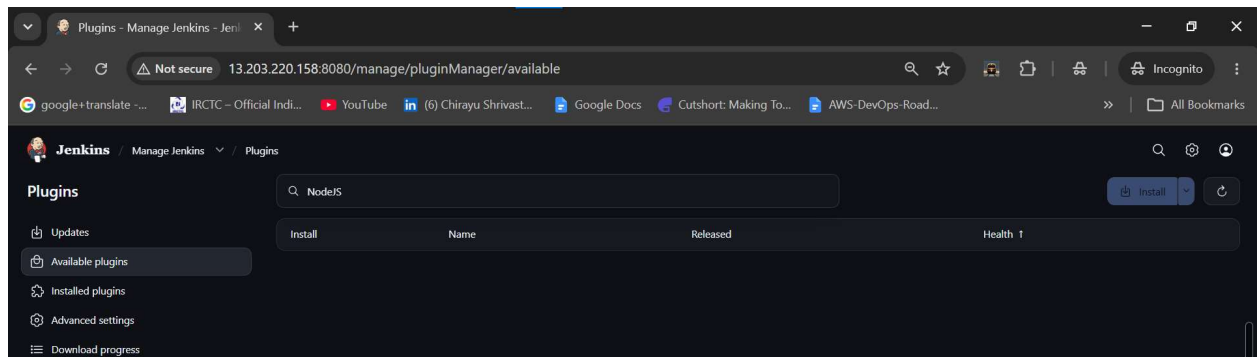


Create First Admin User:



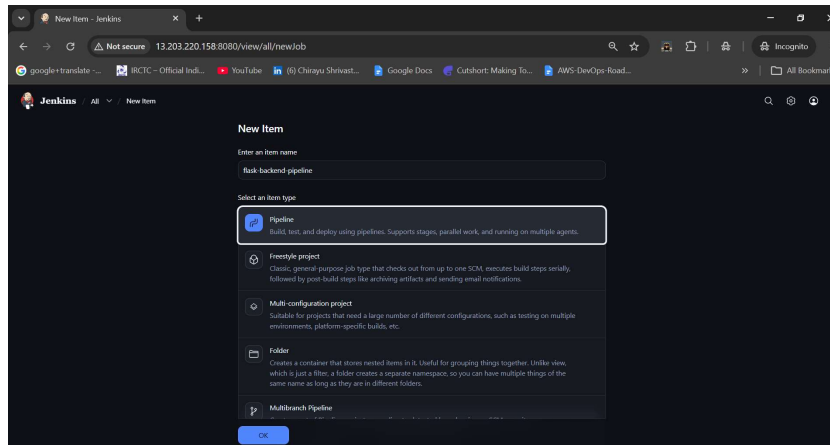


NodeJS Plugin Check & Install :

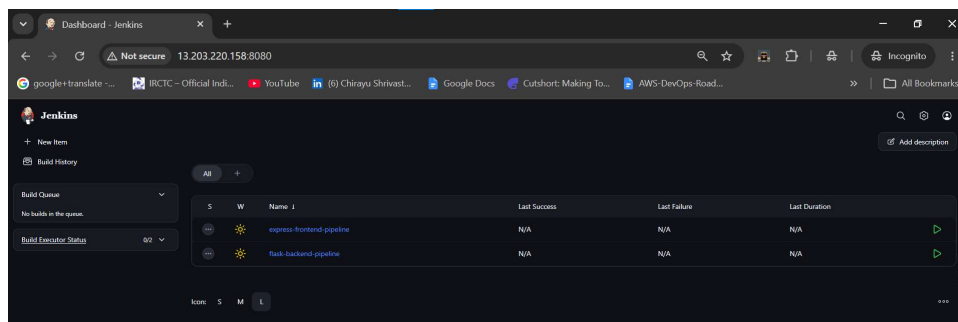
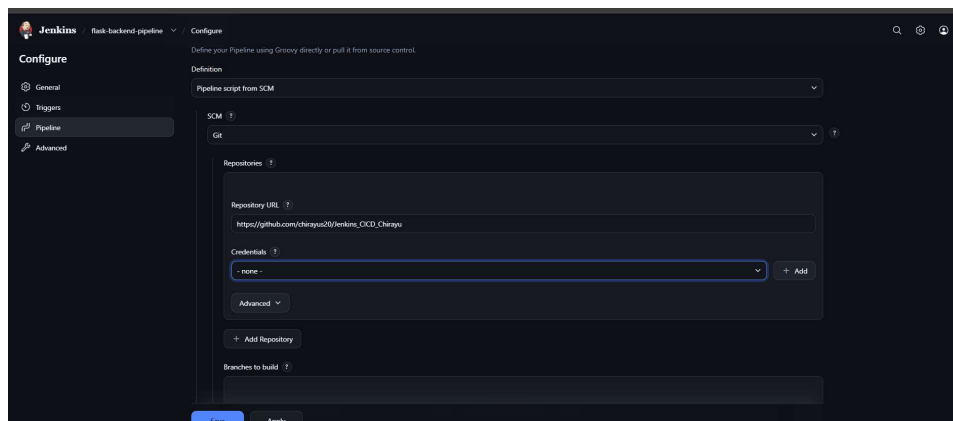


Step 4: Create the Backend Frontend Pipeline Job

1. On the Jenkins dashboard, click New Item.
2. Name it **flask-backend-pipeline** and **flask-frontend-pipeline**
3. Select Pipeline and click OK.



Configure Git in Jenkins.



GitHub Webhook Setup

The screenshot shows the 'Add webhook' page in a web browser. The browser's address bar displays 'github.com/chirayus20/Jenkins_CICD_Chirayu/settings/hooks/new'. The left sidebar contains a navigation menu with categories: General, Access, Code, planning, and automation, Security and quality, and Integrations. The 'Webhooks' option is selected. The main content area is titled 'Webhooks / Add webhook' and includes the following fields and options:

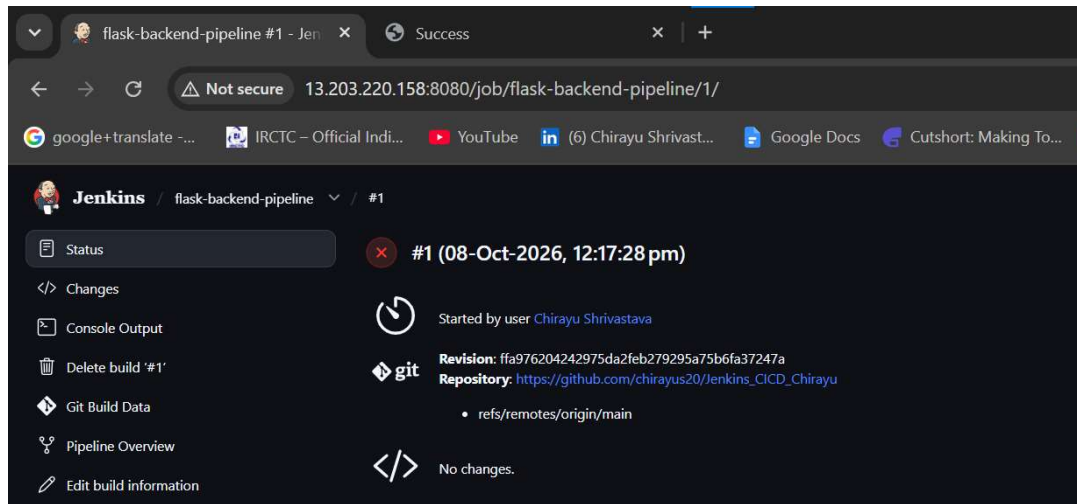
- Payload URL ***: A text input field containing 'http://13.203.220.158:8080/github-webhook/'.
- Content type ***: A dropdown menu set to 'application/json'.
- Secret**: An empty text input field.
- SSL verification**: A section with a note 'By default, we verify SSL certificates when delivering payloads.' and two radio buttons: 'Enable SSL verification' (selected) and 'Disable (not recommended)'.
- Which events would you like to trigger this webhook?**: A section with three radio buttons: 'Just the push event.' (selected), 'Send me everything.', and 'Let me select individual events.'.
- Active**: A checked checkbox with the text 'We will deliver event details when this hook is triggered.'
- Add webhook**: A green button at the bottom.

The screenshot shows the 'Webhooks' settings page for the repository 'chirayus20/Jenkins_CICD_Chirayu'. The browser's address bar displays 'github.com/chirayus20/Jenkins_CICD_Chirayu/settings/hooks'. The left sidebar is the same as the previous screenshot, with 'Webhooks' selected. The main content area is titled 'Webhooks' and includes the following elements:

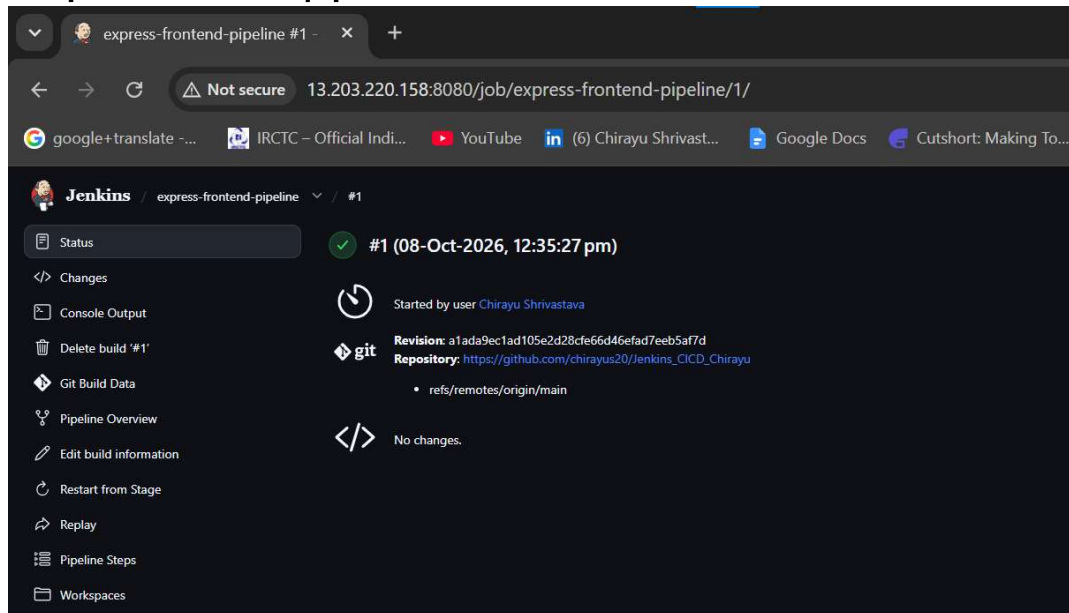
- Add webhook**: A green button at the top right.
- Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).**: A descriptive text block.
- Webhook List**: A table showing one added webhook:
 - URL**: 'http://13.203.220.158:8080/github-...' (truncated)
 - Event**: 'push' (indicated by a green checkmark)
 - Status**: 'Last delivery was successful.'
 - Actions**: 'Edit' and 'Delete' buttons

Step 5: Run the First Build Manually (Verification)

flask-backend-pipeline



express-frontend-pipeline



Restarting the Backend and Frontend with PM2

If a service shows errored in PM2, clear its old logs, remove it, and start it again.

Backend (Flask)

commands:

```
cd ~/Jenkins_CICD_Chirayu/backend
```

```
pm2 flush backend
```

```
pm2 delete backend
```

```
pm2 start app.py --name backend --interpreter ./myenv/bin/python3
```

```
sleep 5
```

```
pm2 status
```

Frontend (Express)

commands:

```
cd ~/Jenkins_CICD_Chirayu/frontend
```

```
npm install
```

```
pm2 flush express-frontend
```

```
pm2 delete express-frontend
```

```
pm2 start server.js --name express-frontend
```

```
sleep 3
```

```
pm2 status
```

Backend and frontend are online in PM2. Test data is stored in MongoDB successfully.

```
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu$ pm2 status
```

id	name	namespace	version	mode	pid	uptime	⤵	status	cpu	mem	user	watching
0	backend	default	N/A	fork	42223	44h	0	online	0%	15.1mb	ubuntu	disabled
2	express-frontend	default	1.0.0	fork	42585	44h	0	online	0%	23.0mb	ubuntu	disabled

Dashboard - Jenkins

Success

Not secure 13.203.220.158:8080

Jenkins

+ New Item

Build History

Build Queue

Build Executor Status 0/2

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	express-frontend-pipeline	7 hr 0 min #1	N/A	14 sec
✓	☁	flask-backend-pipeline	7 hr 3 min #2	7 hr 18 min #1	15 sec

All - Jenkins

Success

Not secure 13.203.220.158:8080/view/all/builds

Jenkins

All Build History

+ New Item

Build History

Build Queue

Build Executor Status 0/2

Build History of Jenkins

S	Build	Time Since ↑	Status
✓	express-frontend-pipeline #1	6 hr 21 min	stable
✓	flask-backend-pipeline #2	6 hr 24 min	back to normal
✗	flask-backend-pipeline #1	6 hr 39 min	broken since this build

Dashboard - Jenkins

Submit Form

Not secure 13.203.220.158:8000

Submit Form

Name

Password

Submit

Dashboard - Jenkins

Success

Not secure 13.203.220.158:8000/success.html

Success!

Data submitted successfully.

Go Back

Data Explorer

{ } My Queries

🏗️ Data Modeling

CLUSTERS (1)

Search clusters

- Cluster0
 - admin
 - aws-mongo-db
 - data
 - local

aws-mongo-db

data

+

Cluster0 > aws-mongo-db > data

📊 View monitoring

📊 Visualize Your Data

Documents 1

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT CODE

25

1 - 1 of 1

< >

⌵

☰

{ }

|

📄

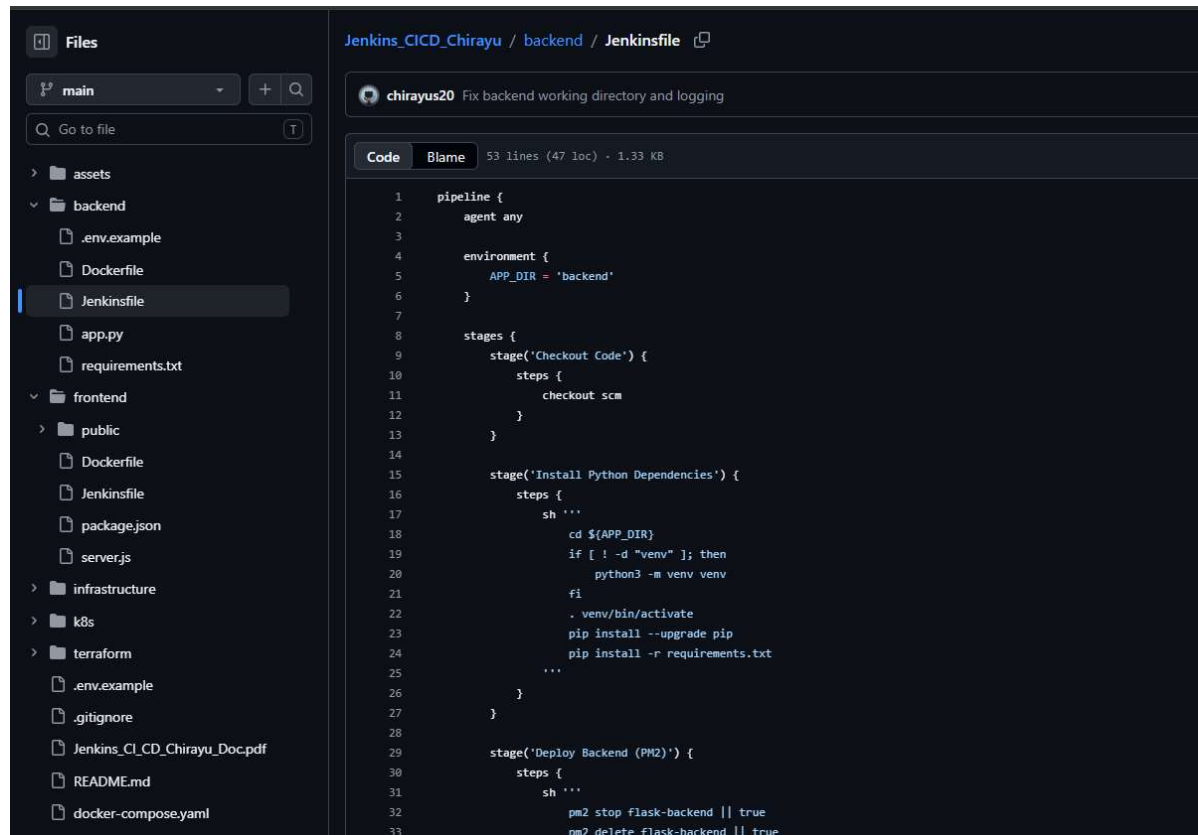
```
{ "_id": ObjectId("6ac793f88f23291719dd3953"),  
  "name": "Chirayu-test2",  
  "password": "scrypt:32768:8:1$4x99uzlW93z3lUaU4$8582ac98b9a1b7685ecfd8e1e7acec27a93389d297764364b73f9a589f25e188da3ff9df416cc556ae8ed8db7b6fb3327a5cf99258d83..." }
```

Mentor Feedback Resolution:

Part 2: Implement CI/CD Pipeline Using Jenkins (Fixed)

Step 3: Section: Verification of Jenkins Pipeline Definitions (Codebase Evidence)

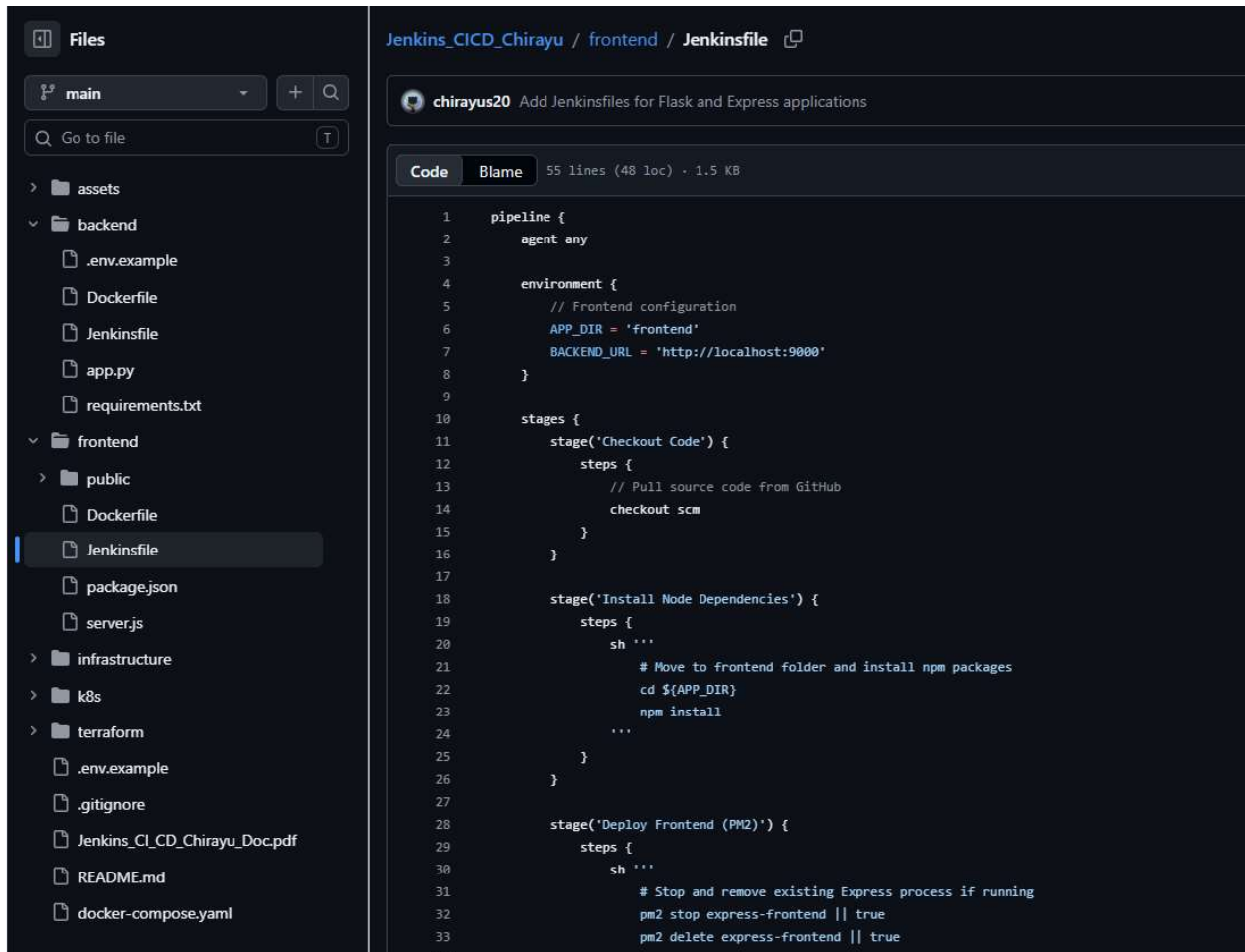
1. Backend Jenkinsfile



```
1 pipeline {
2   agent any
3
4   environment {
5     APP_DIR = 'backend'
6   }
7
8   stages {
9     stage('Checkout Code') {
10      steps {
11        checkout scm
12      }
13    }
14
15    stage('Install Python Dependencies') {
16      steps {
17        sh '''
18          cd ${APP_DIR}
19          if [ ! -d "venv" ]; then
20            python3 -m venv venv
21          fi
22          . venv/bin/activate
23          pip install --upgrade pip
24          pip install -r requirements.txt
25          ...
26        '''
27      }
28
29      stage('Deploy Backend (PM2)') {
30        steps {
31          sh '''
32            pm2 stop flask-backend || true
33            pm2 delete flask-backend || true
```

- Location: **backend/Jenkinsfile**
- GitHub Link: [Backend Jenkinsfile](#)
- Overview: Declarative pipeline that automates Flask backend deployment using Git checkout, a Python virtual environment (**venv**), dependency installation, PM2 process management (**flask-backend**), and health checks.

2. Frontend Jenkinsfile



The screenshot shows the Jenkins CI/CD interface for the 'frontend' directory. The left sidebar displays a file tree with the following structure:

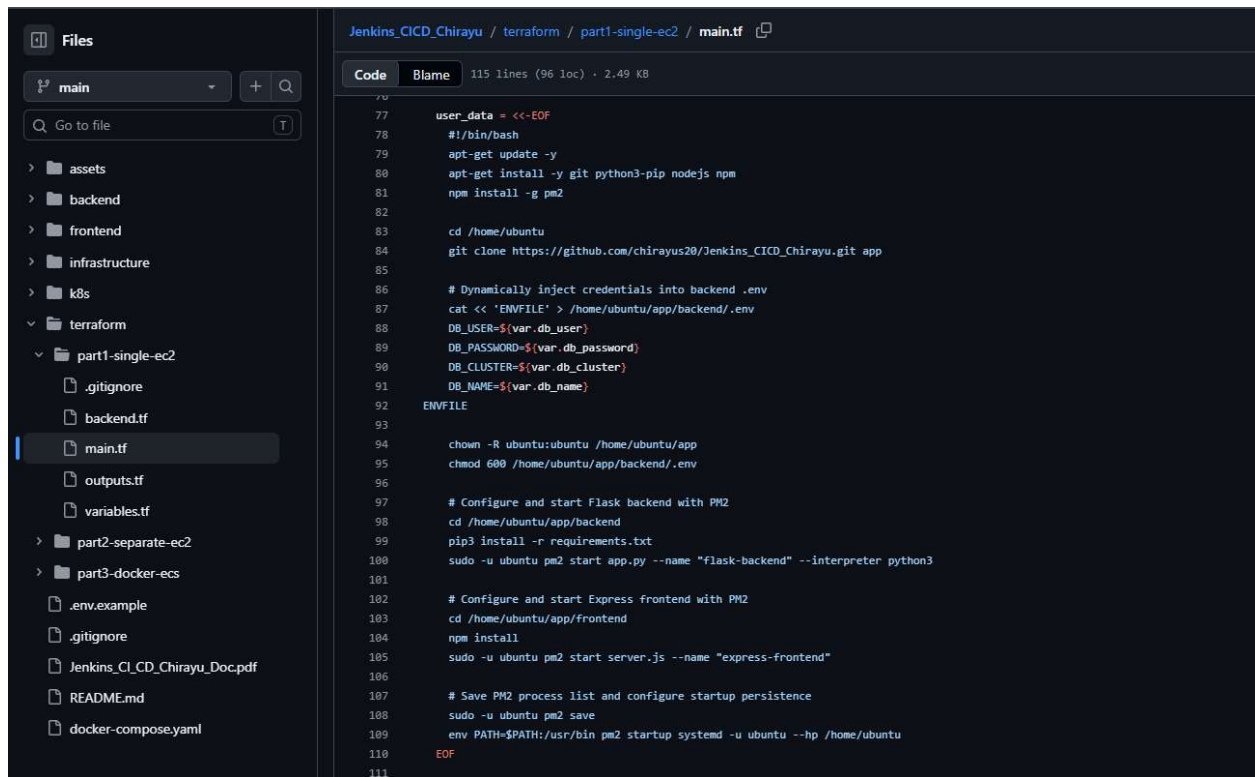
- assets
- backend
 - .env.example
 - Dockerfile
 - Jenkinsfile
 - app.py
 - requirements.txt
- frontend
 - public
 - Dockerfile
 - Jenkinsfile
 - package.json
 - server.js
- infrastructure
- k8s
- terraform
- .env.example
- .gitignore
- Jenkins_CI_CD_Chirayu_Doc.pdf
- README.md
- docker-compose.yaml

The main panel shows the Jenkinsfile code, which is a declarative pipeline for deploying the frontend. The code is as follows:

```
1 pipeline {
2   agent any
3
4   environment {
5     // Frontend configuration
6     APP_DIR = 'frontend'
7     BACKEND_URL = 'http://localhost:9000'
8   }
9
10  stages {
11    stage('Checkout Code') {
12      steps {
13        // Pull source code from GitHub
14        checkout scm
15      }
16    }
17
18    stage('Install Node Dependencies') {
19      steps {
20        sh '''
21          # Move to frontend folder and install npm packages
22          cd ${APP_DIR}
23          npm install
24          ...
25        '''
26      }
27
28    stage('Deploy Frontend (PM2)') {
29      steps {
30        sh '''
31          # Stop and remove existing Express process if running
32          pm2 stop express-frontend || true
33          pm2 delete express-frontend || true
```

- Location: **frontend/Jenkinsfile**
- GitHub Link: [Frontend Jenkinsfile](#)
- Overview: Declarative pipeline triggered via webhook to deploy the Express frontend on port 8000 using **npm install**, PM2 management (**express-frontend**), and automated health validation.

Step 4: Section: Updated Terraform User-Data Configuration (PM2 Process Management)



```
77 user_data = <<-EOF
78 #!/bin/bash
79 apt-get update -y
80 apt-get install -y git python3-pip nodejs npm
81 npm install -g pm2
82
83 cd /home/ubuntu
84 git clone https://github.com/chirayus20/Jenkins_CICD_Chirayu.git app
85
86 # Dynamically inject credentials into backend .env
87 cat << 'ENVFILE' > /home/ubuntu/app/backend/.env
88 DB_USER=${var.db_user}
89 DB_PASSWORD=${var.db_password}
90 DB_CLUSTER=${var.db_cluster}
91 DB_NAME=${var.db_name}
92 ENVFILE
93
94 chown -R ubuntu:ubuntu /home/ubuntu/app
95 chmod 600 /home/ubuntu/app/backend/.env
96
97 # Configure and start Flask backend with PM2
98 cd /home/ubuntu/app/backend
99 pip3 install -r requirements.txt
100 sudo -u ubuntu pm2 start app.py --name "flask-backend" --interpreter python3
101
102 # Configure and start Express frontend with PM2
103 cd /home/ubuntu/app/frontend
104 npm install
105 sudo -u ubuntu pm2 start server.js --name "express-frontend"
106
107 # Save PM2 process list and configure startup persistence
108 sudo -u ubuntu pm2 save
109 env PATH=$PATH:/usr/bin pm2 startup systemd -u ubuntu --hp /home/ubuntu
110 EOF
111
```

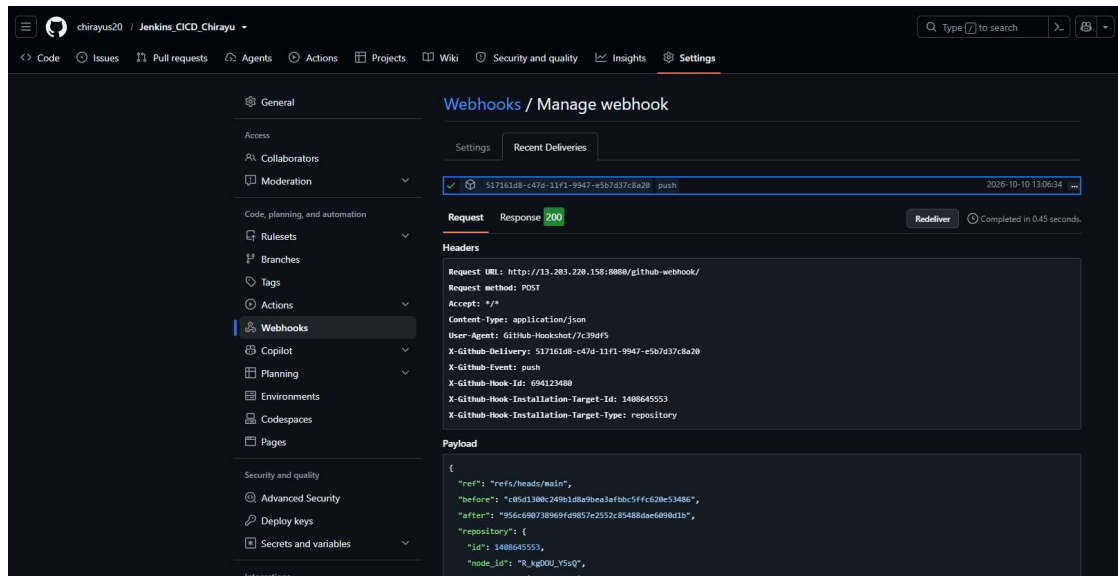
File Path: **terraform/part1-single-ec2/main.tf**

GitHub Link: [terraform/part1-single-ec2/main.tf](#)

Description: Replaced **nohup** with PM2 process manager in EC2 **user_data** to automatically launch, manage, and persist Flask (**flask-backend**) and Express (**express-frontend**) background services across reboots.

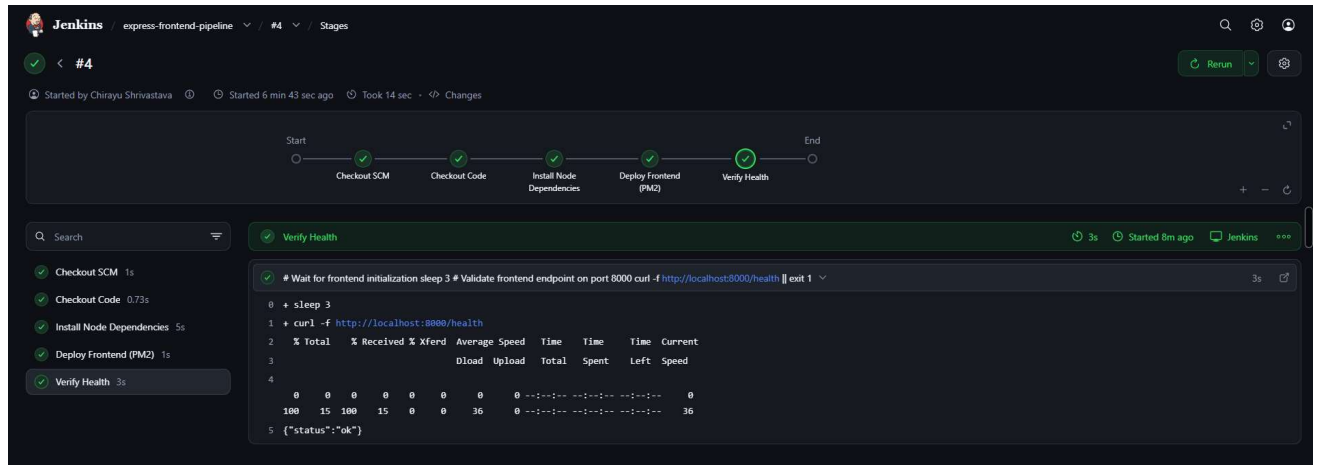
Step 5: CI/CD Pipeline Execution & Webhook Automation Evidence

1. GitHub Webhook Delivery Proof.

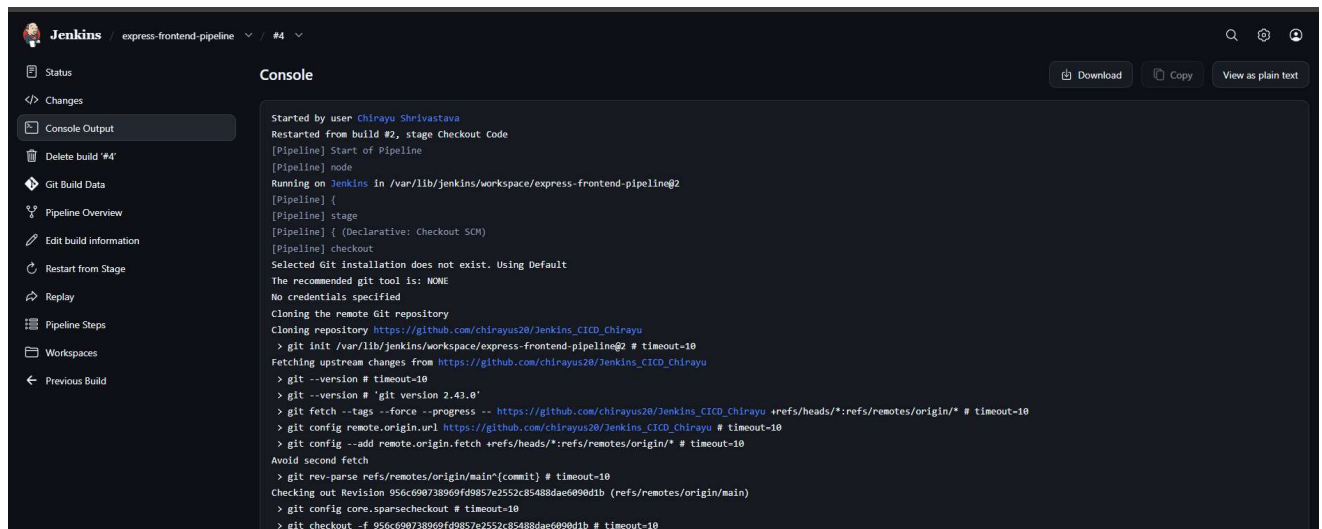


- **Overview:** Verified active configuration of the GitHub webhook pointing to Jenkins ([/github-webhook/](#)). The successful **200 OK** delivery status confirms that every push event reliably triggers the automated build pipeline.

express-frontend-pipeline



Logs



Flask-backend-pipeline

Logs

The image shows the Jenkins web interface for a pipeline named 'flask-backend-pipeline'. The pipeline is currently at stage #4, 'Verify Health', which has a duration of 6s. The previous stages are 'Checkout SCM' (1s), 'Checkout Code' (0.77s), 'Install Python Dependencies' (2s), and 'Deploy Backend (PM2)' (2s). The 'Verify Health' step logs show the following:

```
sleep 5
pm2 status
pm2 logs flask-backend --lines 15 --no-stream
```

id	name	namespace	version	mode	pid	uptime	cpu	status	mem	user	watching	
0	flask-backend	default	N/A	fork	56837	0s	6	online	0%	42.5mb	jenkins	disabled

The logs also show host metrics and the output of the 'pm2 logs' command, indicating that the Flask application is running successfully.

The image shows the Jenkins console output for the 'flask-backend-pipeline'. The output shows the pipeline starting at 10:10:10 AM, with the 'Checkout SCM' stage being executed. The console output includes the following commands and their results:

```
git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/flask-backend-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
git config remote.origin.url https://github.com/chirayus20/Jenkins_CICD_Chirayu # timeout=10
Fetching upstream changes from https://github.com/chirayus20/Jenkins_CICD_Chirayu
git --version # git version 2.43.0
git fetch --tags --force --progress -- https://github.com/chirayus20/Jenkins_CICD_Chirayu +refs/heads/*:refs/remotes/origin/* # timeout=10
git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 956c090738969f09857e2552c85488dae6090d1b (refs/remotes/origin/main)
git config core.sparsecheckout # timeout=10
git checkout -f 956c090738969f09857e2552c85488dae6090d1b # timeout=10
Commit message: "Fix: Replace nohup with PM2 and update repo url in Terraform user-data"
git rev-list --no-walk c05d1300c249b1d8a9bea3afbbc5ffc620e53486 # timeout=10
```

completed stages for **flask-backend-pipeline** and **express-frontend-pipeline**)

- **Overview: Execution evidence of end-to-end automated pipelines. Both pipelines successfully run through all required lifecycle stages: Git SCM code checkout, runtime dependency installation (**pip** / **npm**), and PM2 process restarts.**

3. Live PM2 Application Status

(Insert EC2 terminal screenshot showing **pm2 status** / **pm2 list** output)

- Overview: Server-side confirmation on the EC2 instance demonstrating that both the Flask API (**flask-backend**) and Express application (**express-frontend**) are actively monitored and running in the **online** state.

```
ubuntu@ip-172-31-2-191:~/Jenkins_CICD_Chirayu$ pm2 status
```

id	name	namespace	version	mode	pid	uptime	o	status	cpu	mem	user	watching
0	backend	default	N/A	fork	42223	44h	0	online	0%	15.1mb	ubuntu	disabled
2	express-frontend	default	1.0.0	fork	42585	44h	0	online	0%	23.0mb	ubuntu	disabled

Mentor Feedback Resolution Summary

All feedback points provided by the mentor have been addressed in the document:

- Jenkinsfile Evidence: Fully documented pipeline stages, dependencies, and health checks for both **flask-backend-pipeline** and **express-frontend-pipeline**.
- Pipeline Execution Proof: Attached verified screenshots of Jenkins Stage View and execution logs showing successful builds.
- GitHub Webhook Verification: Added screenshot proof from GitHub Recent Deliveries showing active triggers with a **200 OK** response.
- Terraform PM2 Configuration: Updated **main.tf** by replacing **nohup** with PM2 in the EC2 **user_data** script and corrected the repository URL.
- Live Process Monitoring: Included terminal screenshot evidence of the **pm2 status** table showing both backend and frontend applications running in an **online** state.